

Module 3: AI for Mechatronics

Loss Functions, Optimizers and Training Strategies

Dr. Rajan Prasad

SMEC
VIT University

Fall 2026

Three Main Components

- ➊ **Loss function**
Measures the prediction error.
- ➋ **Optimizer**
Updates the weights and biases.
- ➌ **Training strategy**
Improves stability and generalization.

Basic Training Cycle

- ➊ Perform forward propagation
- ➋ Calculate the loss
- ➌ Backpropagate the gradients
- ➍ Update the model parameters
- ➎ Repeat for several epochs

Input → Prediction → Loss → Gradient → Update

The loss defines what the network should learn, while the optimizer determines how the parameters are updated.

Mean Squared Error

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

- Strongly penalizes large errors
- Smooth and easy to optimize
- Sensitive to outliers

Mean Absolute Error

$$\mathcal{L}_{\text{MAE}} = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|$$

- Penalizes errors linearly
- More robust to outliers
- Not differentiable at zero

Mechatronics Applications

Temperature prediction, force estimation, energy-consumption forecasting, remaining useful life and robot-position prediction.

Use MSE when large errors must be penalized strongly; use MAE when robustness to outliers is important.

Binary Cross-Entropy

Used for two-class problems:

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

Output activation: Sigmoid

Categorical Cross-Entropy

Used for multiclass problems:

$$\mathcal{L}_{\text{CCE}} = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C y_{ic} \log(\hat{y}_{ic})$$

Output activation: Softmax

Mechatronics Applications

Binary fault detection uses BCE, while classification of multiple machine-fault categories uses categorical cross-entropy.

The loss function must match both the learning task and the output-layer activation.

Choose the Loss According to the Learning Task

Common Choices

Learning task	Suitable loss
Numerical prediction	MSE or MAE
Binary classification	Binary cross-entropy
Multiclass classification	Categorical cross-entropy
Autoencoder reconstruction	MSE or BCE
Imbalanced classification	Weighted or focal loss

Autoencoder Example

The input is also the target:

$$\mathbf{y} = \mathbf{x}$$

The reconstruction loss is

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^N (x_i - \hat{x}_i)^2$$

The network learns to reproduce the input accurately.

A mathematically valid loss may still be unsuitable if it does not represent the objective of the application.

Gradient-Descent Update

For a model parameter θ :

$$\theta_{t+1} = \theta_t - \eta \frac{\partial \mathcal{L}}{\partial \theta_t}$$

- η : learning rate
- $\nabla \mathcal{L}$: direction of increasing loss
- $-\nabla \mathcal{L}$: direction of decreasing loss

Mini-Batch SGD

The gradient is estimated using a mini-batch $\mathcal{B}$:

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} \mathcal{L}_{\mathcal{B}}$$

- Efficient for large datasets
- Produces frequent parameter updates
- Gradient estimates may fluctuate

A small learning rate causes slow training, whereas a very large learning rate may prevent convergence.

SGD with Momentum

Momentum accumulates previous gradients:

$$\mathbf{v}_t = \beta \mathbf{v}_{t-1} + \mathbf{g}_t$$

$$\theta_{t+1} = \theta_t - \eta \mathbf{v}_t$$

- Reduces oscillations
- Accelerates consistent movement
- Uses previous update directions

RMSProp

RMSProp tracks squared gradients:

$$\mathbf{s}_t = \beta \mathbf{s}_{t-1} + (1 - \beta) \mathbf{g}_t^2$$

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\mathbf{s}_t} + \epsilon} \mathbf{g}_t$$

- Adapts the step size for each parameter
- Handles gradients with different scales

Momentum uses the history of gradient direction, whereas RMSProp adapts updates using gradient magnitude.

Adam Combines Momentum and Adaptive Learning Rate

Adam Update

First-moment estimate:

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1) \mathbf{g}_t$$

Second-moment estimate:

$$\mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2) \mathbf{g}_t^2$$

Parameter update:

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon}$$

Why Is Adam Popular?

- Uses adaptive learning rates
- Often converges quickly
- Works well with noisy gradients
- Requires limited initial tuning

Important

Adam is a strong starting choice, but it does not guarantee the best validation performance for every problem.

Adam combines the main ideas of momentum and RMSProp within one optimizer.

Normalization Within a Mini-Batch

For an activation x_i :

$$\mu_B = \frac{1}{m} \sum_{i=1}^m x_i$$

$$\sigma_B^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu_B)^2$$

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$$

$$y_i = \gamma \hat{x}_i + \beta$$

Benefits

- Stabilizes intermediate activations
- Often allows larger learning rates
- Can accelerate training
- Reduces sensitivity to initialization

Typical Position

Linear/Convolution $\rightarrow$ BatchNorm $\rightarrow$ Activation

During inference, batch normalization uses stored running statistics instead of the current mini-batch statistics.

During Training

A random binary mask is applied:

$$r_i \sim \text{Bernoulli}(1 - p)$$

$$\tilde{h}_i = \frac{r_i h_i}{1 - p}$$

- p : dropout probability
- Selected neuron outputs become zero
- A new mask is generated during training

Why Does It Help?

- Reduces co-adaptation among neurons
- Limits overfitting
- Encourages distributed feature learning
- Acts approximately like an ensemble

During Inference

Dropout is disabled, and all neurons participate in the prediction.

An excessively large dropout probability can remove too much information and cause underfitting.

Early-Stopping Procedure

- 1 Train the model for one epoch
- 2 Calculate the validation loss
- 3 Save the best model parameters
- 4 Check whether validation loss improves
- 5 Stop after no improvement for several epochs

The permitted waiting period is called **patience**.

Combined Training Strategy

- 1 Select a task-appropriate loss
- 2 Begin with a suitable optimizer
- 3 Monitor training and validation loss
- 4 Use batch normalization when appropriate
- 5 Add dropout when overfitting occurs
- 6 Apply early stopping

$$t_{\text{best}} = \arg \min_t \mathcal{L}_{\text{validation}}^{(t)}$$

The objective is not simply to minimize training loss, but to achieve low prediction error on unseen data.

Instructions

- Scan the QR code to access the quiz.
- Sign in using your **VIT Google account**.
- The quiz contains **10 questions**.
- Select the most appropriate answer.
- Only **one submission per account** is allowed.

QUIZ: Scan to Begin



Training Strategies Control the ??

Thank You